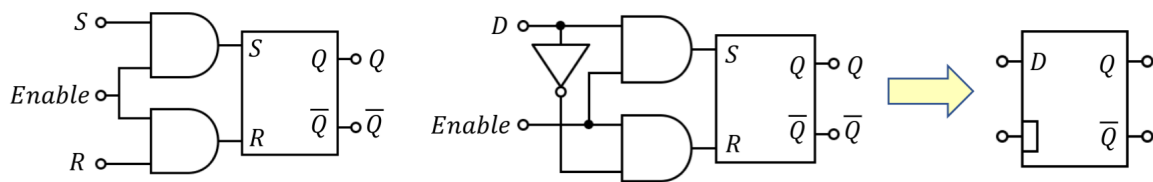


1 Lesson 30-09-26

Last lesson we finished up on sequential circuits, introducing the *bistable circuit* and making it controllable via a pair of NOR ports (implementing an *SR latch*). Let's start again from there.

SR latch w/ enable: let's add enable circuitry to the SR latch. Note that *enable* here doesn't mean setting the output to high impedance (as in tristates), but making the network unresponsive to changes in the inputs.

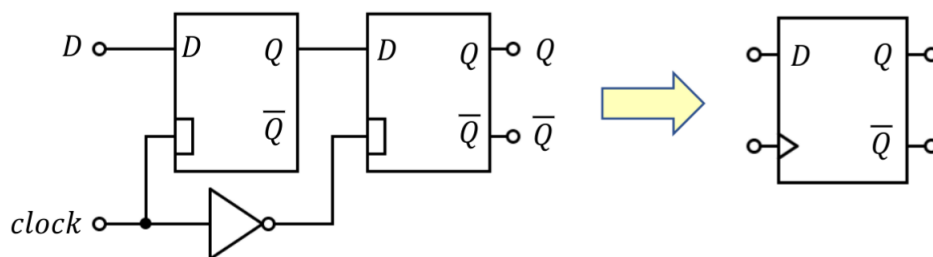
The circuits we could get are the following:



D latch: let's add enable circuitry to the SR latch.

Note that we decided to simplify the SR latch circuitry in the middle circuit, by setting S to a data value D , and R to its negation. This allows us to build what's essentially a **D latch** (last circuit in the row).

D flip flop: this is one last memory circuit we can build, which is even more convenient than the past two.



In the figure we note the master/slave implementation, but there are many others that can be built. The idea of the D flip flop, no matter the implementation, is that the output value Q follows the input D *only* on the rising edges of the clock signal. This makes D flip flops non *transparent*: a change in the input isn't transmitted directly to the output (but only transiently during clock rising edges): in other words, if the circuit oscillates, it oscillates synchronized to the clock. This makes this circuit extremely useful in sequential networks as a memory element (for instance in *registers*).

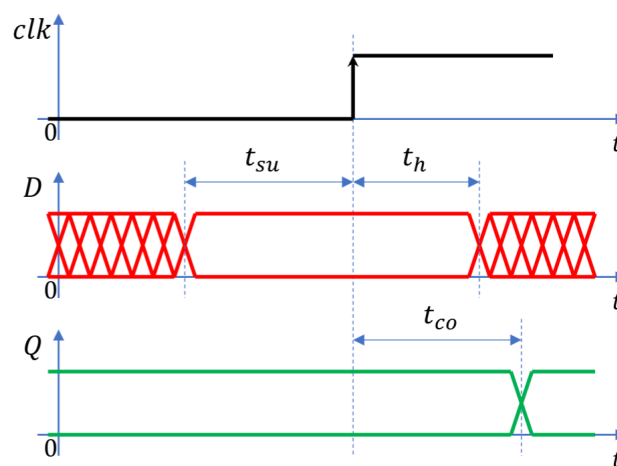
Other characteristics: let's note other characteristics of sequential networks.

Firstly, when talking about flip-flops and latches, we note that there are usually other inputs provided:

- **Asynchronous clear (or reset):** sets the output to 0, without waiting the active clock edge;
- **Asynchronous set:** sets the output to 1, without waiting the active clock edge;
- **Clock enable:** masks the clock input.

These signals are often active low.

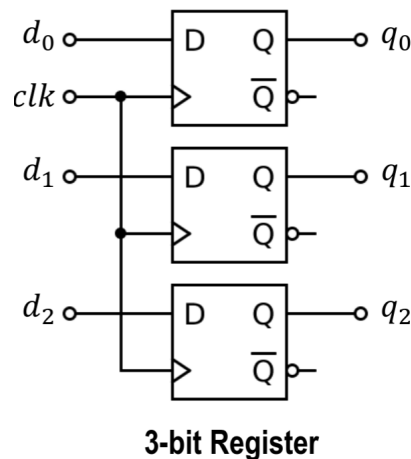
Other important characteristics are the **timing specifications** of flip-flops:



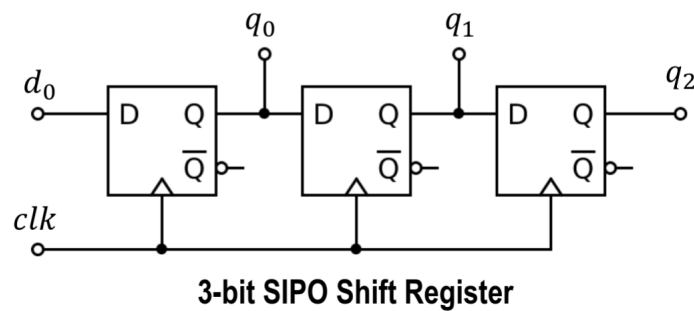
- **Setup** time: the time the data input has to be stable *before* the rising edge, for the flip-flop to correctly recognize its value;
- **Hold** time: the time the data input has to be stable *after* the rising edge, for the flip-flop to correctly recognize its value;
- **Clock to output** time: the time the flip-flop takes to propagate its input to its output after a clock rising edge.

These values are often in the order of microseconds, and directly determine the maximum clock speeds we can achieve.

Registers: we introduced them, now let's look at them. A **register** in the electronics sense is nothing more than a combination of multiple flip-flops with a common clock and different data inputs. This allows registers to store whole *words* of data, of arbitrary size. Let's look at a 3 bit register as an example.



Shift register: a **shift register**, often called a *delay line*, is like a register which takes in a single bit of data and propagates it through a set of flip-flops.



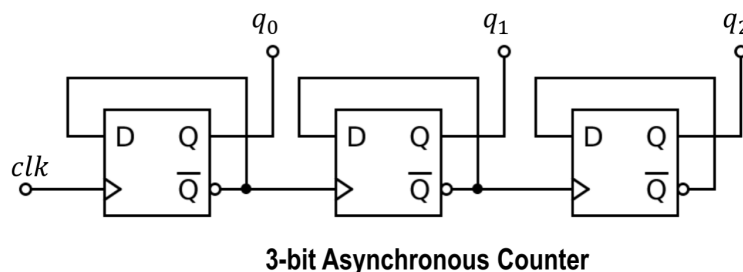
There are different kinds:

- Serial In, Parallel Out (**SIPO**);
- Parallel In, Serial Out (**PISO**): these need extra control inputs such as:
 - Parallel Load (PL)
 - Load Enable (LE)
- Serial In, Serial Out (**SISO**): these often come with parallel out, as well.

All shift registers may have a shift enable signal.

Counter: there are 2 kinds of counters:

- **Asynchronous counter:** the clock propagates from one FF to the next. This means that the most significant bits are updated later than least significant bits. This is undesirable as it entails *data races* along the FFs. Still, it is the one we show an example of as it's the simplest;
- **Synchronous counter:** the same clock is provided to all FFs. Internal logic used to compute the input to each FF: this means all output bits are updated at the same time.



This being said, there are other characteristics which determine the behavior of a counter:

- Up or Up/Down counters;
- Binary or Decade counters;
- Parallel Load to initialize counter value;
- Carry input and output to cascade counters;
- Asynchronous reset.

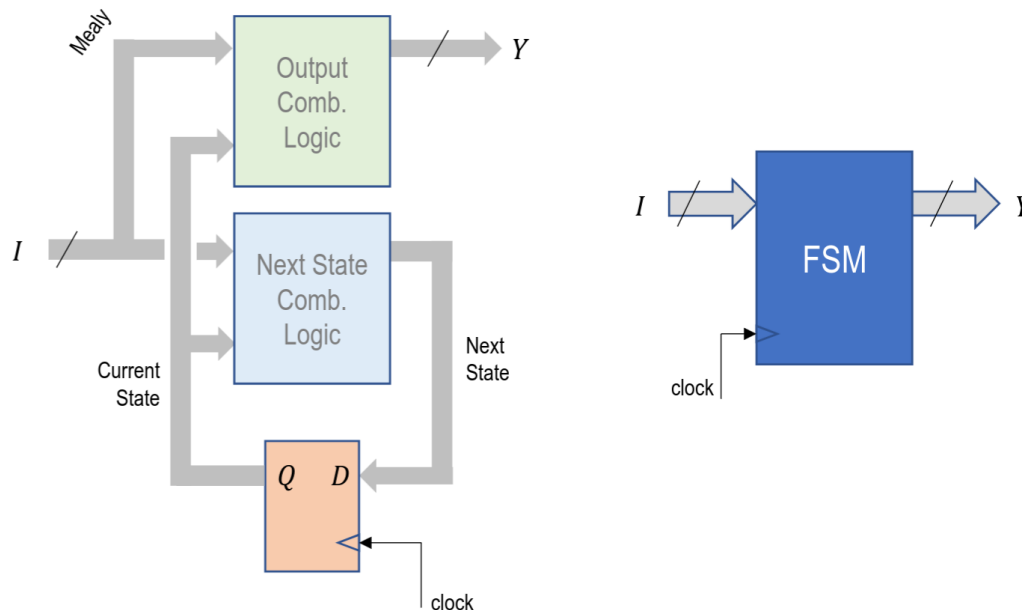
FSM: an **FSM** (*Finite State Machine*) is a graph structure consisting of *nodes*, corresponding to **states**, and *edges*, corresponding to **state transitions**. This way, a state machine defines all possible states of a digital systems and all the ways states can transition one to another.

A FSM description can also entail a network's outputs:

- Edges may have output values (*Mealy* model);
- States may have output values (*Moore* model);
- Transitions are synchronized with an external clock signals (*retarded* or *synchronized* Mealy and Moore models).

State machines can formally be described via two truth tables:

- One for *state transitions* (next state), with respect to inputs and current state
- One for *outputs*, with respect to current state (Moore) and inputs (Mealy).



To implement a state machine as a circuit we first need the *specification* of that state machine, which we said is either given by truth tables or state transition graphs. Then, we can encode the internal state in a special register (the **STAR**, or *STAtus Register*). The STAR register has to be as big as the base-2 logarithm of the state space (in *binary* encoding, as big as the state space for *one-hot* encoding).

At this point the two truth tables become two combinational logic circuits:

- One that takes in the current state (Moore) and/or the current inputs (Mealy) and returns the output of the network;
- One that takes the current state and the current inputs to determine the next state (which gets fed back to the STAR register).

1.1 VHDL

VHDL (*VHSIC Hardware Description Language*) is a hardware description language that can model the behavior and structure of digital systems at multiple levels of abstraction, ranging from the system level down to that of logic gates.

- It is used to *describe* the structure and the behavior of a **digital system**;
- The description can be used to *simulate* and *synthesize* a **digital circuit**.

The nice part of VHDL is that the syntax is similar to a programming language.

The basic design element of a VHDL description is an **entity**, which is a design unit comprised of:

- The **entity** specification, which acts as a black-box interface for the component:
 - **Name** of entity;
 - **Inputs**;
 - **Outputs**;
 - **In/out** ports;
- The **architecture** specification, which describes the entity's behavior and/or internal structure. VHDL allows for a number of specification styles:
 - **RTL** (*Register Transfer Level*): uses *concurrent signal assignments*.
These are *boolean expressions* assigned to a signal, executed in *parallel*, so without any implied order. Not that lists of concurrent assignments are *not* sequential in execution. High level operators such as arithmetic, logic and conditionals can be used;
 - **Structural**: similar to a schematic diagram, but using a textual programming language.
We first declare *components*, then *signals* to interconnect those components, and lastly *map* signal and input/output ports to components. This allows for a hierarchical description of a circuit (Top/Down or Bottom/Up);
 - **Behavioral**: uses a process, which is a sort of an endless loop, specified in a programming language-like way.
In most cases it uses an implicit *wait statement* on a *sensitivity list*:
 - * The process suspends when it reaches the end
 - * Resumes when any signal on the sensitivity list changesInstructions in the process are, this time, in fact *sequential*, but may still be implemented in parallel to increase performance if the compiler finds it is feasible. Still, we note that signal assignments happen only *at the end* of a certain process. Higher level constructs such as if or case conditional expressions are available, variables can be defined and assignments to variables are immediate.

1.2 Introduction to embedded systems

This course and the entire degree program are focused on **embedded systems**, so it might be natural to ask *what* an embedded system even is. An embedded system is an electronic system, that is a system made up of:

- Hardware components;
- Software components;
- Communication systems.

designed to be part of a larger system, not necessarily electronic:

- Vehicles (cars, planes, ...);
- Industrial systems;
- Home appliances.

This means that an embedded system:

- Is, in fact, **embedded**, i.e. it lives in a larger environment;
- It should **react**, typically in real-time, to external stimuli;

- It is dedicated to a particular **application** (single task).

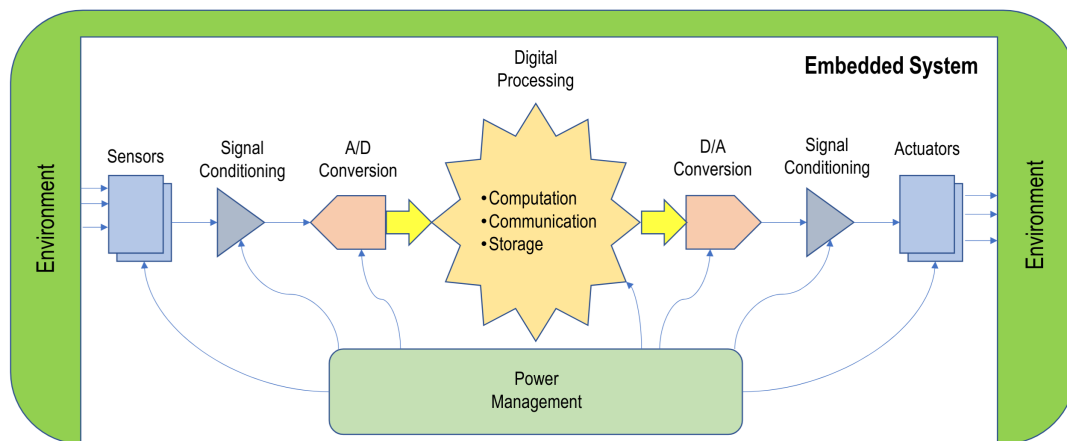
This gives designers the opportunity to highly optimize and adapt to different situations, in highly constrained environments.

Designing embedded systems, in this regard, is difficult because:

- The design space is extremely large;
- It requires skills from many very different domains;
- It requires to simultaneously optimize numerous design metrics.

1.2.1 Architecture of an embedded system

Let us look at the typical components of an embedded system:



- We already know that an embedded system lives inside an **environment**, from which it *senses* data and which it modifies by *acting* on it;
- Data will be obtained from **sensors** that return electrical values proportional to physical quantities in the environment;
- Data will be **conditioned**, which means *amplified*, *filtered*, or anyways manipulated in order to be more useful to the embedded system;
- Data will be converted from an analog signal to a digital value ready for computation by an **A/D converter**;
- The central part of the embedded system performs **digital processing** of the recovered data from the sensors, signal conditioning and A/D conversion parts. What comes out of this phase is digital signals which will be used to control the *actuators*;
- Data will be converted from the digital processing stage back into the analog domain by a **D/A converter**;
- Again, we will have some amount of **conditioning** on the output signal;
- The conditioned signal will drive **actuators**, which can actually affect the environment;
- All of these components need **power management** to be correctly powered by some power supply (often batteries, but also wall power, ...).

1.2.2 Design metrics for embedded systems

A **design metric** is a measurable feature of a system implementation. Common metrics are:

- **Unit cost** U : the monetary cost of manufacturing each copy of the system;
- **NRE cost** N : one-time monetary cost of developing (designing) the system;
- **Total cost** T : total cost of producing N_{unit} tests. Calculated as $T = N + U \times N_{unit}$;
- **Per-product cost** p : cost of a singular product, with amortized NRE costs. Calculated as $p = \frac{T}{N_{unit}} = \frac{N}{N_{unit}} + U$. Keeping the per-product cost low entails spending as little as possible on NRE (as these costs get amortized over units produced) or producing as many units as possible. Of course the unit cost U plays a significant role: lowering is sometimes useful at the cost of increasing N ;
- **Time-to-market**: it is the time required to develop a product to the point it can be sold to customers. Smaller time-to-market means less losses due to delayed market entry (for instance, when competing with other products);
- **Performance**: it can be measured in many ways. For instance:
 - **Latency**, or *response time*, the time between task start and task end;
 - **Throughput**, tasks per second, usually $\frac{1}{\text{Latency}}$.

Parallelism is a way to increase performance. Usually, N parallel blocks allow to increase throughput to $N \times \text{Throughput}_{\text{Serial}}$ (unless Ahmdal's law comes into play);
- **Technologies**, that is technical processes, methods and knowledge, are in some way a design metric.

1.2.3 Technologies

Let's look at a few **technologies** which we have available for the design of embedded systems. We noted the design space is very large, so it is useful to know about them.

Processor technology

Let's look at **processor** technologies.

- The *General Purpose (GP)* processor, as found in desktop PCs, is the classic programmable micro-processor we are used to.
 - They are not optimized for any specific application;
 - They are often cheaper than general purpose technology: billions of units are sold every year;
 - Flexible, low time-to-market: software is easy to update and modify;
 - No high performance compared to a dedicated design;
 - Not power efficient
- *Embedded* micro-processors, though, are different from PC micro-processors. For instance, they are usually within a *System-On-Chip (SoC)*, which includes needed components and a number of peripherals.
- A method of implementing embedded processors is *Application Specific Instruction Processor (ASIP)*. This entails:
 - A general purpose processor with specific hardware for a certain application domain, for instance **DSP** (*Digital Signal Processing*);
 - Dedicated to signal processing application;
 - Possibly dedicated hardware to drive and read external pins;
 - Cheap;
 - Flexible, low time-to-market;

- Higher performance and better power than a GP processor.
- Another method is *Single Purpose* (custom) *Processor*. This entails:
 - Digital hardware designed specifically for a certain application;
 - Best performance;
 - Best power consumption;
 - Small size;
 - Long time-to-market;
 - Very high NRE costs;
 - Low flexibility;
 - Good for large volume productions and for extremely high performance designs.

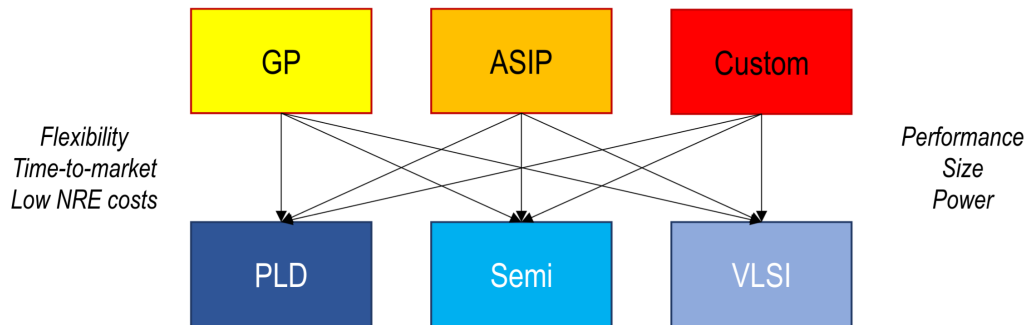
IC technology

ICs (*Integrated Circuits*) also have different technologies with different use cases.

- *Full Custom* (**VLSI**, *Very Large Scale Integration*):
 - The integrated circuit is entirely designed and optimized for a particular application;
 - Transistor sizing, placing and routing;
 - Best for performance, size and power consumption;
 - High NRE costs;
 - All masks in the production process are dedicated to a single design;
 - Long time-to-market;
 - Currently used to design micro-processors (GPs and ASIPs), memories and off-the-shelf Programmable Logic.
- *Semi-Custom* (Gate Array / Standard cells) technology:
 - The lower layers are fully or partially built: design entails placing of large blocks, routing;
 - Good performance, size and power consumption;
 - Lower NRE compared to a full custom implementation;
 - Some masks are shared among more than one design;
 - Still, high NRE costs (tough lower than VLSI);
 - Long time-to-market.
- *Programmable Logic Device* (**PLD**):
 - The integrated circuit is fully built by the manufacturer: the embedded system designer buys the IC. The PLD can be customized after fabrication for a specific application;
 - Logic functions and interconnections are programmable;
 - Low NRE costs;
 - Fairly good performance and power consumption, but worse than full custom and semi-custom;
 - High unit cost;
 - Large size.

Orthogonality

Processor and IC technologies are **orthogonal**: the design space is very large, and takes very long to explore. We show a figure showing the different processor and IC technologies we considered, and their characteristic (low N vs high U , or low U vs high N).



Early design decisions are difficult, and it is useful to make use of modern design technologies to evaluate different solutions.

It can also be of use to note that the design phase of a product is also a field where more than one methodology (i.e. a technology and its associated techniques) can be used. This is particularly important in companies, which might have long historical experience with one methodology instead of another. Using well-trusted methodologies or using new ones is also an important factor to consider.

1.3 Memories

Memories are the first component of embedded systems we will encounter. They exist in the digital domain, and more specifically in the storage component of the digital processing part of an embedded system.

In general, there are different kinds of memory, which differ in types, operations available, interfaces, timings and composition of different memories to make bigger ones.

We know the usual types of memory:

- **ROM** (*Read Only Memory*), that is EPROMs, ...
- **RAM** (*Random Access Memory*), that is SRAM, DRAM, ...

We will also note the use of memories as *cache memories* and *memory controllers*.